

Řízení lineárního kyvadla v předmětu ARI

Jáchym Kouba

11. května 2026

1. Úvod

V rámci semestrální práce předmětu ARI jsem si zvolil projekt týkající se řízení lineárního kyvadla. K dispozici mám matematicky popsany model kyvadla a jeho simulaci v Matlabu a Simulinku. Konkrétní cíle úlohy jsem za průběhu semestru diskutoval s profesorem Martinem Hromčíkem, nakonec jsem své cíle stanovil na dva cíle: řízení polohy vozíku se stabilizací kyvadla v dolní poloze a stabilizaci inverzního kyvadla.

2. Popis kyvadla

Pro popis kyvadla jsem měl k dispozici dva zdroje - pdf soubor s matematickým popisem lineárního kyvadla a kód v Matlabu s předpřipravenou simulací v Simulinku.

2.1. Matematické zadání

Uvedu zde matematický popis kyvadla ze zadání, který budu dále používat [1]:

Laboratorní model Kyvadlo na vozíku je nelineární stabilní (pro naše účely) systém s jedním vstupem

- napětí na motoru vyvolávajícím sílu F působící na vozík, u [V] (akční veličina)

a dvěma výstupy:

- úhel natočení kyvadla měřený od vodorovné polohy φ [°],
- poloha vozíku x [m].

Systém kyvadla na vozíku je popsán následujícími diferenciálními rovnicemi

$$(M + m)\ddot{x}(t) - ml\ddot{\varphi}(t)\sin\varphi(t) - ml\dot{\varphi}^2(t)\cos\varphi(t) + b\dot{x}(t) = F(t)$$
$$J_p\ddot{\varphi}(t) + mgl\cos(\varphi(t)) - ml\ddot{x}(t)\sin(\varphi(t)) + 2\delta\dot{\varphi}(t) = 0.$$

Tyto rovnice je možné přepsat do vhodnějšího tvaru

$$\ddot{x}(t) = \frac{1}{f_1(t)} \left[l^2 m^2 g \cos\varphi(t) \sin\varphi(t) - J_p m l \cos\varphi(t) \dot{\varphi}^2(t) \right. \\ \left. + 2\delta m l \sin\varphi(t) \dot{\varphi}(t) - F(t) J_p + J_p b \dot{x}(t) \right]$$
$$\ddot{\varphi}(t) = \frac{1}{f_2(t)} \left[2\delta \dot{\varphi}(t) + m l g \cos\varphi(t) - \frac{m^2 l^2}{M + m} \sin\varphi(t) \cos\varphi(t) \dot{\varphi}^2(t) \right. \\ \left. - \frac{m l}{M + m} \sin\varphi(t) F(t) + \frac{m l b}{M + m} \sin\varphi(t) \dot{x}(t) \right],$$

kde funkce $f_1(t) = -J_p m - J_p M + l^2 m^2 \sin^2\varphi(t)$ a $f_2(t) = -J_p + \frac{l^2 m^2 \sin^2\varphi(t)}{M + m}$.

Parametry v rovnicích jsou: m [kg] je hmotnost kyvadla, M [kg] je hmotnost vozíku, l [m] je délka kyvadla, J_p [kg m² s⁻²] [sic] je moment setrvačnosti kyvadla vzhledem k ose otáčení, g [m s⁻²] je gravitační zrychlení, b [kg s⁻¹] reprezentuje všechna tření úměrná rychlosti vozíku, δ [kg m² s⁻¹] je koeficient viskózního tření v kloubu kyvadla.

2.2. Kontrola zadání

Prvním krokem mé práce bylo ověřit, zda se zadáné rovnice v obou souborech shodují. Obě rovnice z Matlabu jsem přepsal do stejné formy, jako jsou napsané v pdf zadání, viz obr. 1.

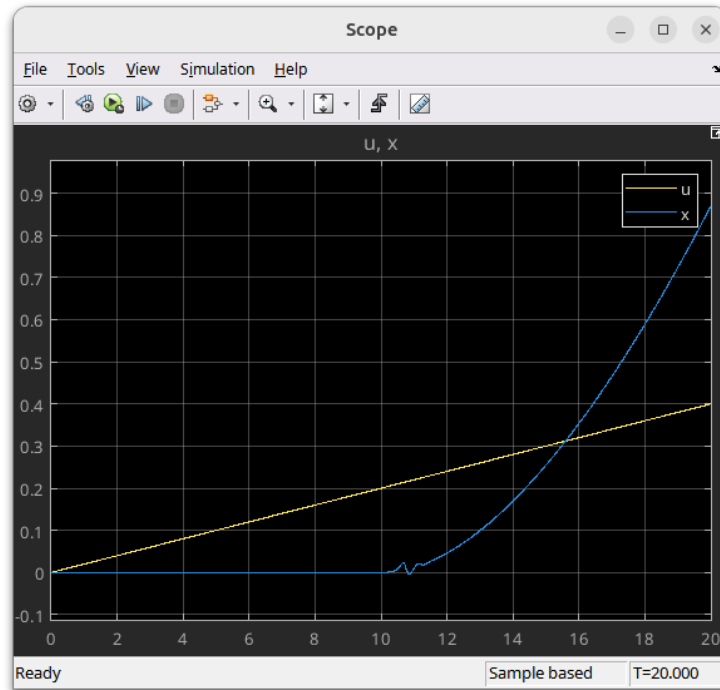
The handwritten derivation on grid paper shows the simplification of a complex equation for acceleration. The derivation starts with a large fraction involving trigonometric functions and derivatives, and through several steps of algebraic manipulation, it simplifies to a final expression for acceleration. The steps involve canceling terms, using trigonometric identities, and simplifying the denominator. The final result is a sum of terms involving the input signal and its derivatives, with coefficients that depend on system parameters like mass, length, and gravity.

Obrázek 1: Kontrola rovnic

Rovnice vypadají po přepsání stejně, až na to, že v Matlabu se vyskytuje $2l$ všude, kde je v pdf napsáno l . Dále budu pokračovat s údaji z Matlabu. Ještě jsem si všimnul, že v pdf zadání se nachází drobná chyba v jednotkách pro J_p : setrvačnost musí mít jednotku $[\text{kg m}^2]$.

2.3. Omezení vstupu a výstupu

Pro vstupní napětí je v zadaném programu uvedena saturace: vozík bude přijímat ignál v rozsahu $u \in [-10; 10]$. Dále jsem zjistil hodnotu deadzone - vozík nereaguje na napětí s hodnotou $\text{abs}(u) \leq 0.2$. Test jsem provedl postupným zvyšováním napětí na vstupu, dokud se vozík nezačal pohybovat, viz obrázek 2.



Obrázek 2: Zjišťování deadzone

3. Linearizace

Abych mohl efektivně pracovat s rovnicemi, tak je potřebuji linearizovat v určitých bodech. Pro účely úlohy potřebuji získat linearizované rovnice kyvadla pro $\varphi = -90^\circ$ (kyvadlo míří kolmo dolů) a pro $\varphi = 90^\circ$ (kyvadlo míří kolmo nahoru). Linearizaci pro dolní polohu provedu pomocí Matlabu následovně:

```
X = [v; y; w; phi];
f = [dv; dy; dw; dphi];

%% Evaluate at DOWN Position (Stable: phi = -pi/2)
% Equilibrium states: [v=0; y=0; w=0; phi=-pi/2], u=0
X_down = [0; 0; 0; -pi/2];

A_down = double(subs(A_sym, [X; u], [X_down; u_eq]));
B_down = double(subs(B_sym, [X; u], [X_down; u_eq]));
```

Linearizace pro horní polohu je téměř identická, pouze zvolím úhel $\varphi = \pi/2$.

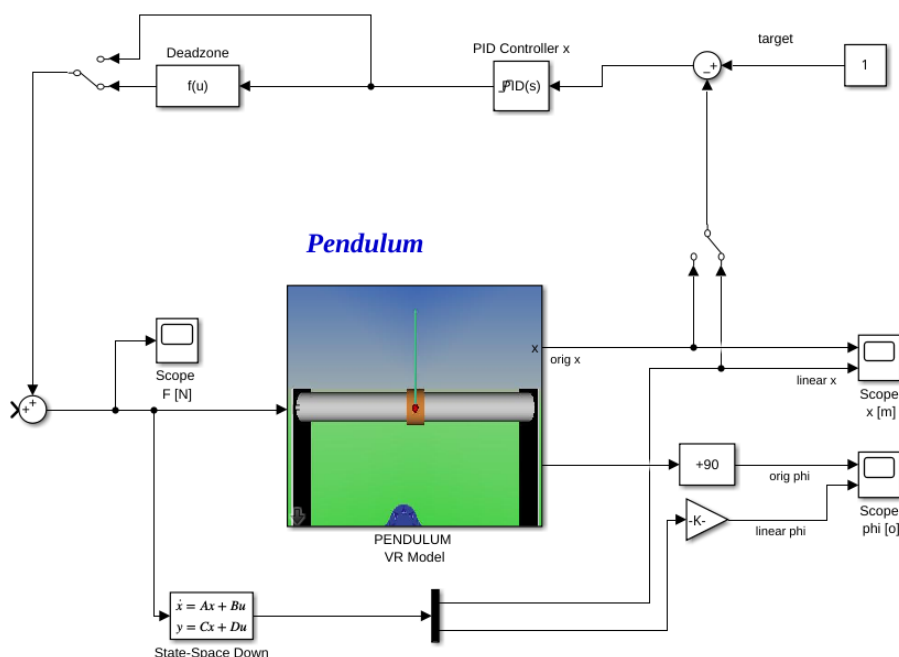
Zobrazením nul jsem si potvrdil, že systém s kyvadlem v dolní poloze je stabilní (má všechny póly v levé polorovině), zatímco systém s kyvadlem v horní poloze je nestabilní (obsahuje jeden pól v bodě 10.56).

4. Řízení polohy vozíku pomocí PID regulátoru

Jako první úlohu jsem si zvolil řízení polohy vozíku - dostanu zadaný bod na ose x a cílem je, aby na něj vozík dojel. Během této fáze budu zcela ignorovat pohyb kvadla.

4.1. Zapojení v Simulinku

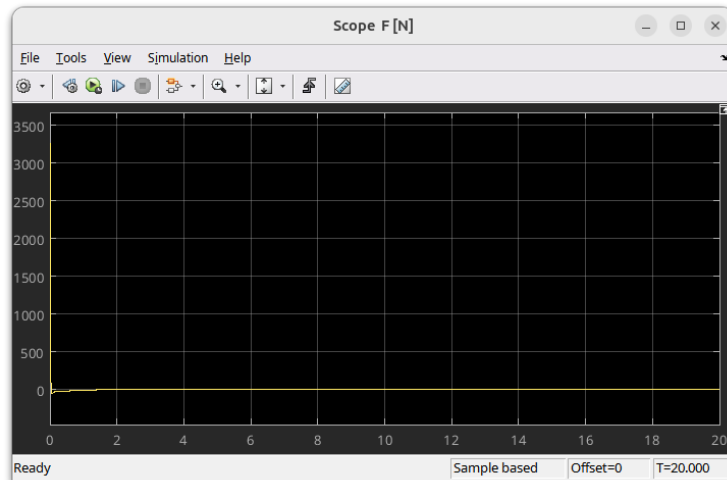
Pro zprovoznění simulace jsem nejdříve zapojil zapojit schéma v Simulinku 3. Linearizaci provádím přes blok **State-Space**, do kterého jsem dal matice **A_down**, **B_down**, **C** a **D**. První dvě matice jsem již zmínil v předchozím textu, pro zbylé platí: $C = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$, $D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. Zajímá mě totiž pouze poloha a úhel, tedy druhá a čtvrtá složka vektoru vstupu.



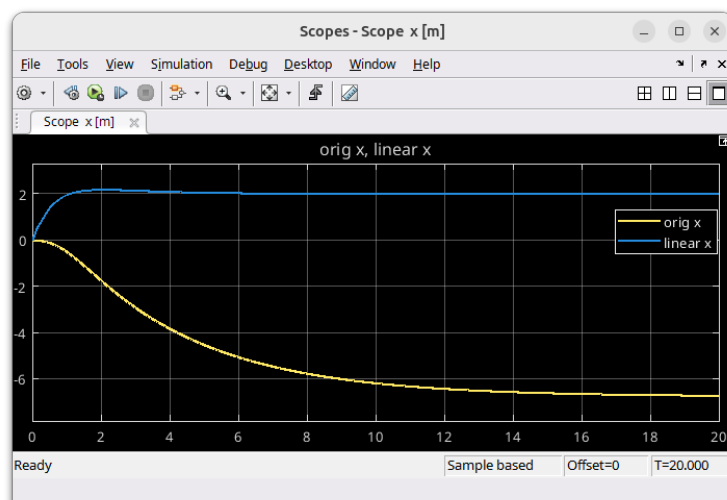
Obrázek 3: Zapojení v Simulinku pro regulátor polohy vozíku

Z linearizovaného modelu vedu signál do Scope pro polohu a pro úhel (rozděluji ho pomocí bloku Demux), úhel musím převést na stupně.

Ve zpětné vazbě mám regulátor, do kterého později dosadím získané hodnoty. Regulátor sám o sobě obsahuje omezení na saturaci a implementaci řešení wind-upu (zvolil jsem metodu clamping - když signál přesahuje hodnotu saturace, tak dále nenačítám I složku regulátoru). Následuje blok, který signál ořízne, pokud se nachází v deadzone. Po přidání těchto bloků jsou nelineární i lineární signál během řízení v dolní poloze téměř shodné, viz 4 a 5.



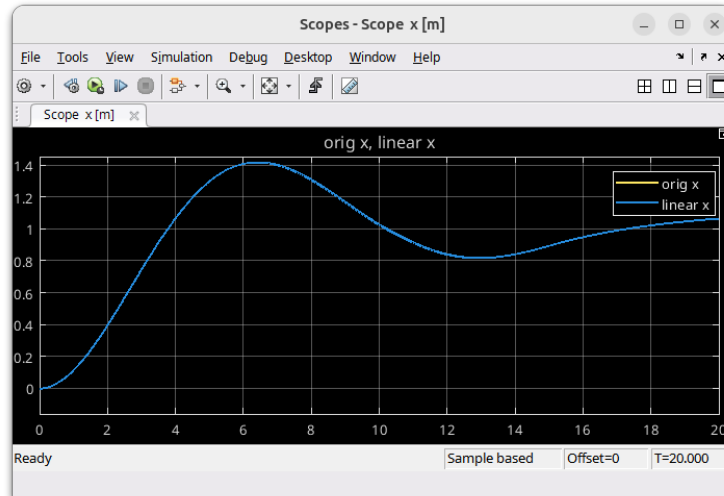
Obrázek 4: Zapojení bez saturace a deadzone - vstup



Obrázek 5: Zapojení bez saturace a deadzone - výstup

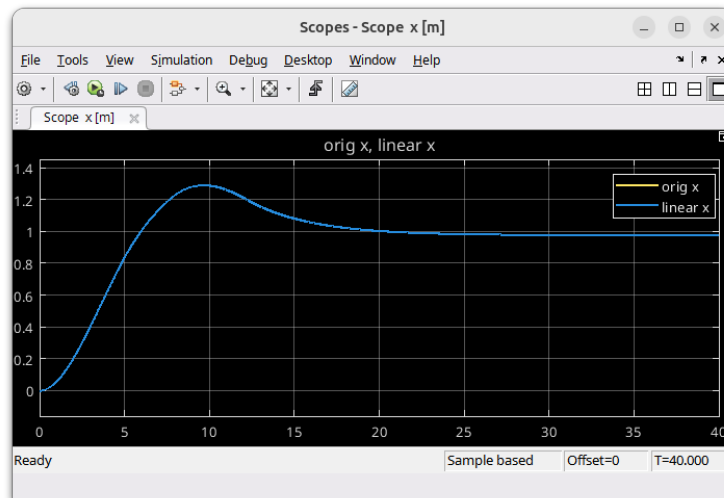
4.2. P regulátor

Nejdříve jsem vyzkoušel řízení pomocí samotného P regulátoru, které ovšem vedlo na mnoho problémů. Při vysoké hodnotě koeficientu K_p docházelo k velkému překmitu a následnému rozkmitání, kvůli kterému se kyvadlo nechtělo ustálit na cílové hodnotě - výsledky jsou na obrázku 6.



Obrázek 6: Překmit P regulátoru

I pro výsledky s kvalitně odladěnou hodnotou K_p nastával klasický problém u P regulátoru, že se kyvadlo neustálí na cílové hodnotě, jako na obrázku 7. Tento jev je navíc výrazně podpořen existencí pásma necitlivosti, které dělá tento problém neřešitelným.

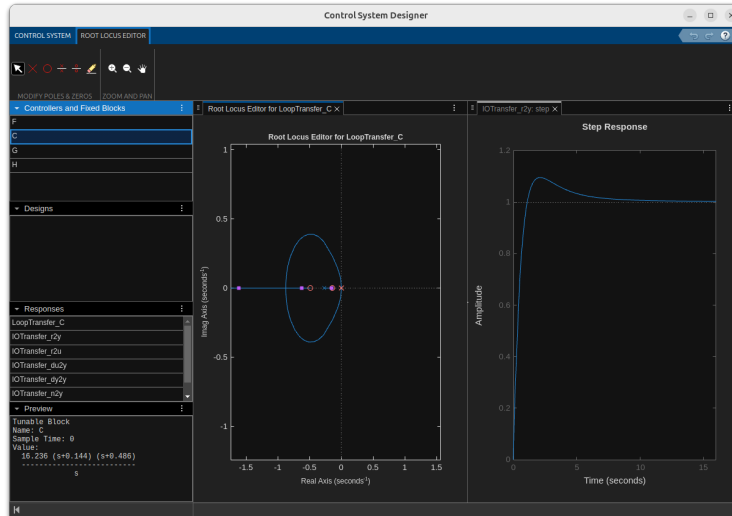


Obrázek 7: Ustálení na špatné hodnotě

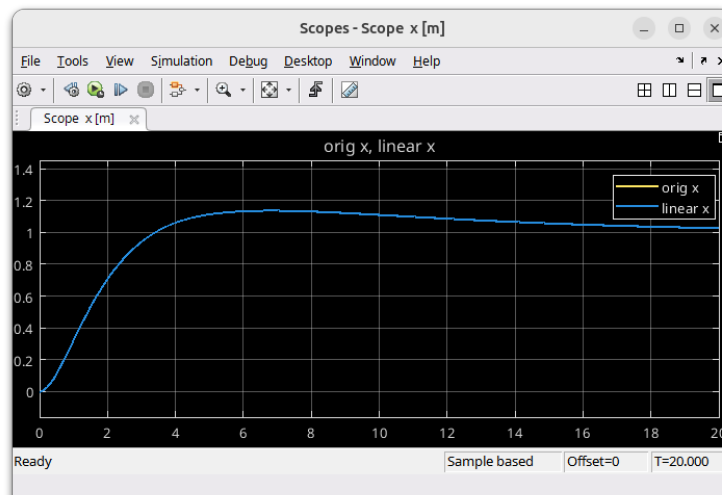
Z výše zmíněných důvodů je nutné implementovat PID regulátor.

4.3. PID regulátor

PID už zvládá vozík regulovat dostatečně dobře. Hodnoty jsem odladil pomocí programu `rltool` 8 a dosadil je do schématu Simulinku. Výsledná regulace na polohu $x = 1$ (na obrázku 9) má překmit menší než 15% a dobu ustálení na 90% reference za 11s.



Obrázek 8: Hledání parametrů PID regulátoru pomocí rltool

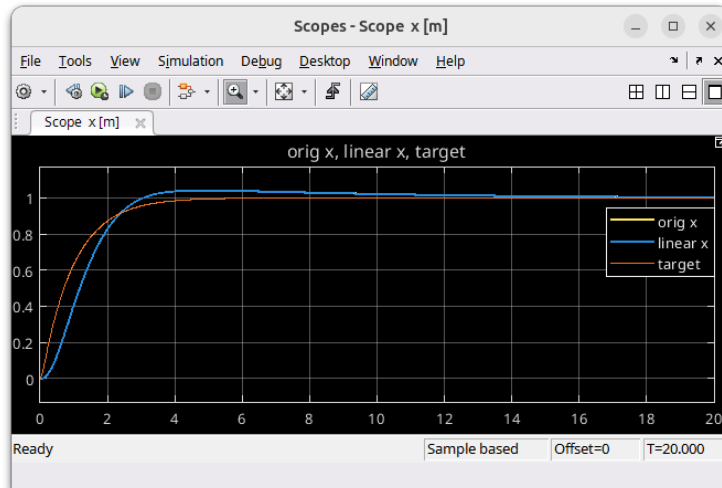


Obrázek 9: Regulace vozíku na polohu $x = 1$

4.4. Model reference

Tato metoda se neprobírala na přednáškách, ovšem po času stráveném na implementaci jsem se rozhodl ji zde ponechat.

Zatím jsem měl jako referenci nastavenou konstantu, jednalo se vlastně o jednotkový skok. Pro lepší řízení mohu ovšem referenci postupně načítat (pomocí přenosu) a tím se vyhnout zbytečně velkému překmitu. Za blok target připojím přenos druhého stupně, jehož hodnoty odladím a dostanu tak efektivnější řízení vozíku. Pro přenos $\frac{1}{0.05s^2 + s + 1}$ dostávám překmit 5% a v rozmezí $\pm 5\%$ od cílové hodnoty bude vozík za 2.6s. Graf je na obrázku 10.



Obrázek 10: Regulace vozíku s modelem reference

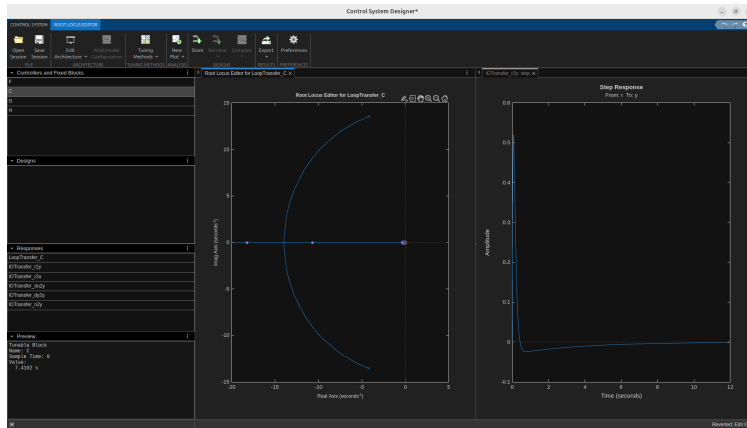
Tímto způsobem, s přenosem $\frac{1}{0.03s^2+2.8s+1}$, jsem dosáhl i řízení polohy vozíku s překmitem méně než 1%, do intervalu 1% od cílové hodnoty se vozík dostane za 9s.

5. Řízení polohy vozíku pomocí PID regulátoru s přida- ným tlumením kmitů

Řízení polohy vozíku už funguje, teď se budu snažit, abych utlumil kmitání kyvadla během pohybu vozíku.

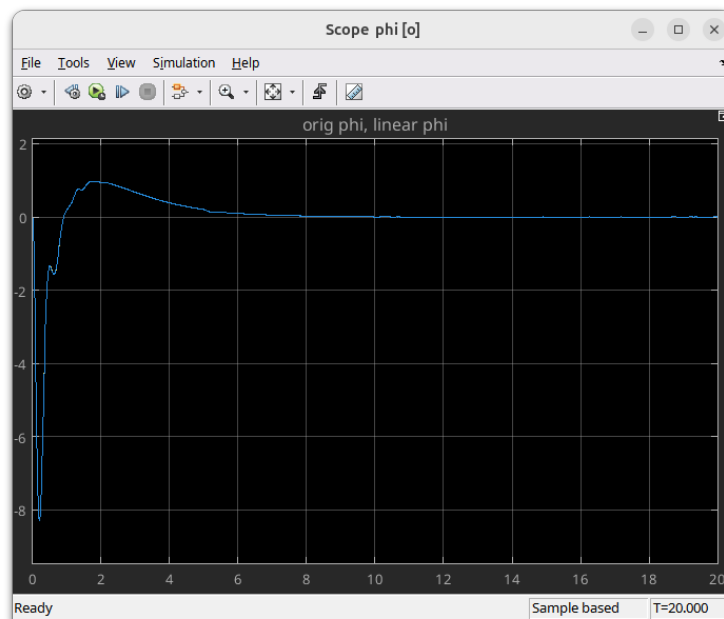
5.1. Tlumení kmitů pomocí D regulátoru

Následujícím cílem mé práce je utlumit kmitání kyvadla během posunu vozíku. Budu testovat PD a D regulátor. Do simulinku si přidám blok regulátoru, do kterého vede jako vstup úhel kyvadla. Jedná se o tlumení, takže výsledná hodnota pro regulaci musí být vždy záporná. Hodnoty regulátoru budu chtít odlatit pomocí programu `rltool` (vkládám systém se záporným znaménkem), pro D regulátor umístím jednu nulu do bodu 0 a budu hýbat s póly, dokud nezískám optimální regulátor na obrázku 11.

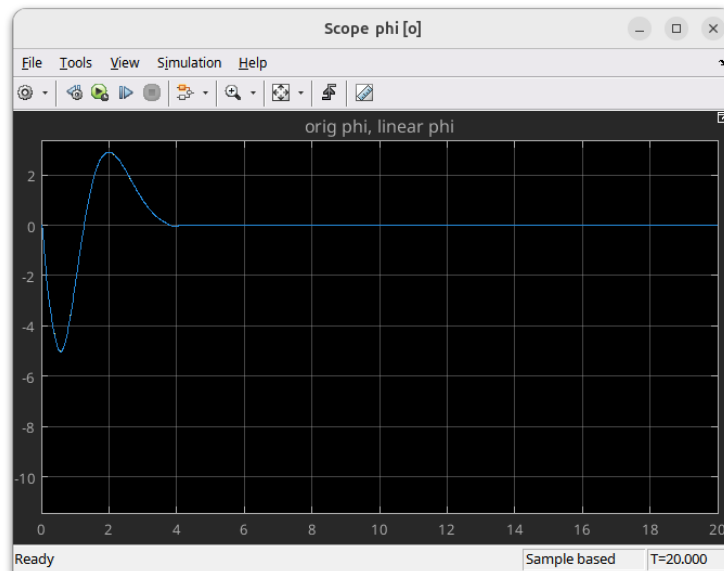


Obrázek 11: Hledání paramterů D regulátoru pomocí rltool

Hodnotou jsem odladil na $k_d = 7.4102$. Regulátorem dojde k úplnému utlumení malých kmitů, tzv. při přesunu vozíku vykoná kyvadlo pouze dva kmity. Zlepšení oproti původnímu stavu 12 je vidět na obrázku 13.



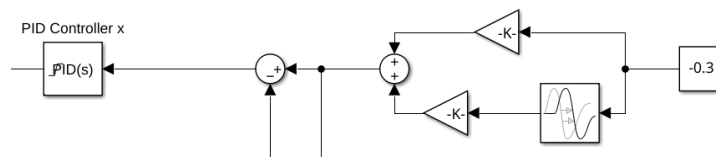
Obrázek 12: Kmitání bez regulátoru



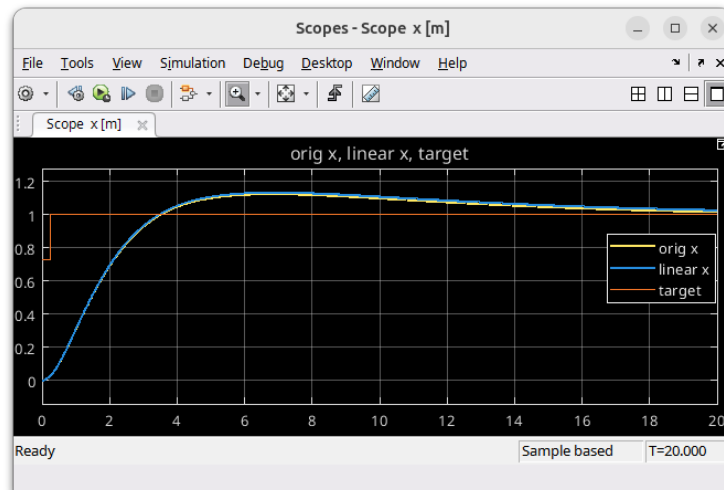
Přidáním P regulátoru bych sice dosáhla zmenšení prvního kmitu o $\sim 20\%$, ale jinak PD regulátor nepřináší žádné výrazné zlepšení, proto ho nebudu implementovat. Vyvýjím tak totiž zbytečně velkou sílu na vozík.

5.2. Posicast

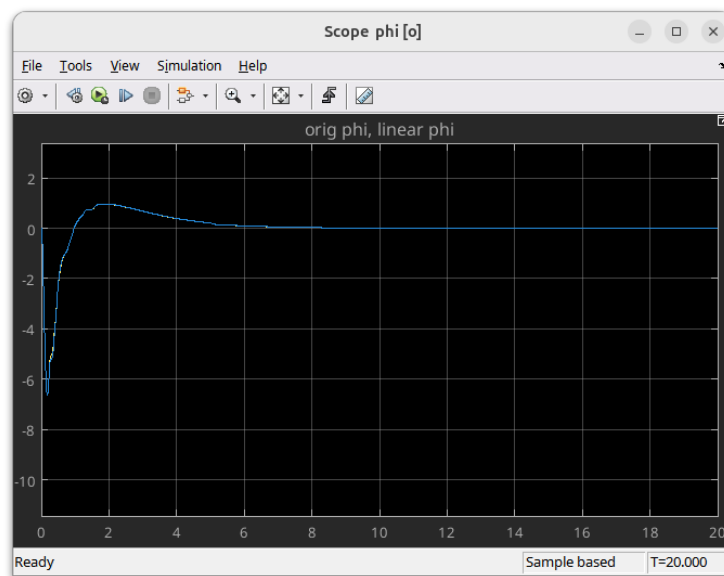
Dále jsem implementoval feedforward řízení posicast. Pomocí funkce `damp()` v Matlabu jsem si našel tlumení a frekvenci komplexně sdružených pólů: $\omega_n = 14.2 \text{ rad} \cdot \text{s}^{-1}$ a $\zeta = 0.297$. Dosadil jsem do vzorců $\tau = \frac{\pi}{\omega_n \sqrt{1-\zeta^2}} = 0.2317 \text{ s}$ a $\Phi = \frac{1}{1+e^{-\tau\zeta\omega_n}} = 0.7265$. Poisicast rozdělí referenční signál na dva skoky, čímž vyruší kmitání kyvadla ve výsledné pozici. Pro implementaci a výsledky posicastu viz obrázky 14 15 16.



Obrázek 14: Implementace posicastu v Simulinku



Obrázek 15: Poloha vozíku s poisicastem



Obrázek 16: Úhel kyvadla s poisicastem

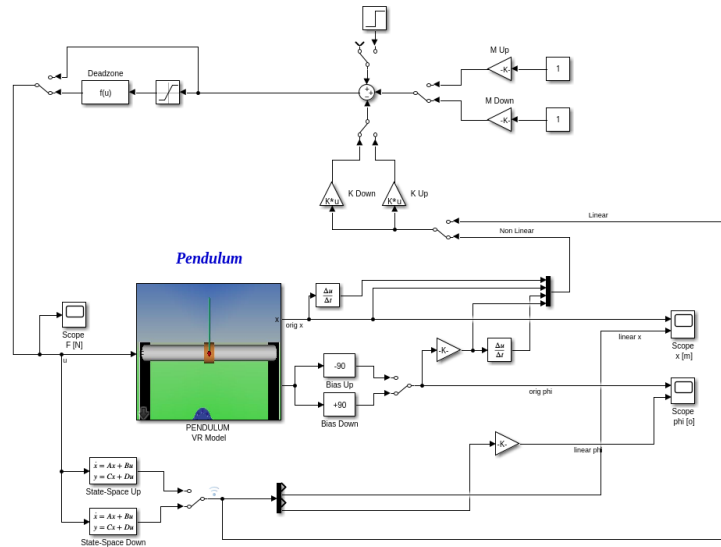
6. Řízení pomocí stavové zpětné vazby

Pomocí stavových metod (moje lineární kyvadlo má 4 stavy - polohu, rychlost, úhel, úhlovou rychlost) bych chtěl implementovat řízení s kyvadlem nejen v dolní, ale i v horní poloze.

6.1. Řízení pomocí stavové zpětné vazby s předfiltrem

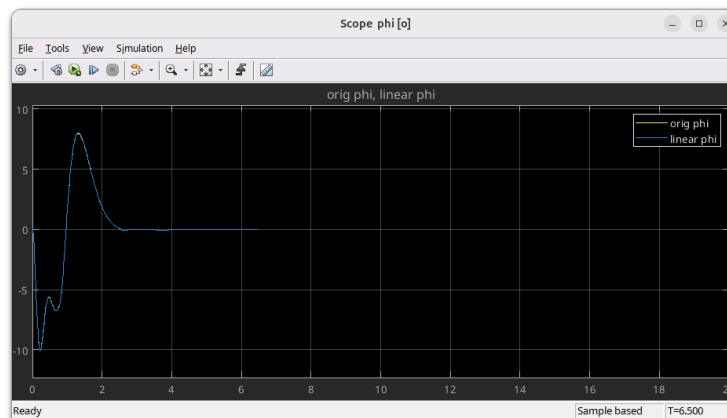
6.1.1. Dolní poloha

Nejdříve vypočítám matici K . Určím (zatím pouze odhadnu) póly na $p = [-3 + 2i, -3 - 2i, -10, -12]$ a matici K dostanu funkcí Matlabu `place`, která ji spočítá dle Ackermanova vzorce. Předfiltr vypočítám pomocí vzorce (s $D = 0$) $M = \frac{-1}{C(A-BK)B}$. Schéma zapojím do Simulinku (obsahuje i regulátor pro horní polohu), viz obrázek 17.

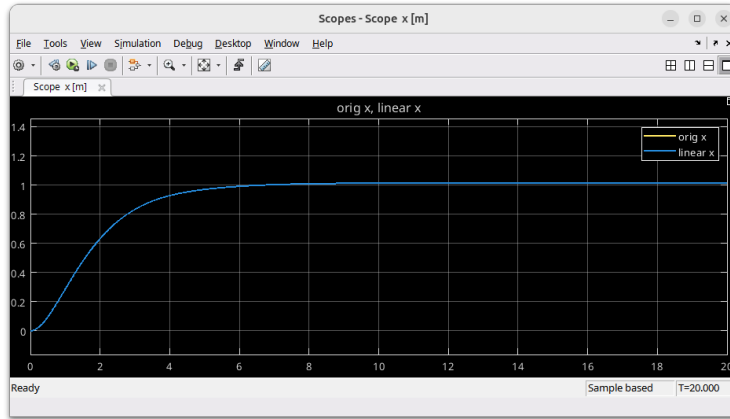


Obrázek 17: Zapojení stavové zpětné vazby s předfiltrem v Simulinku

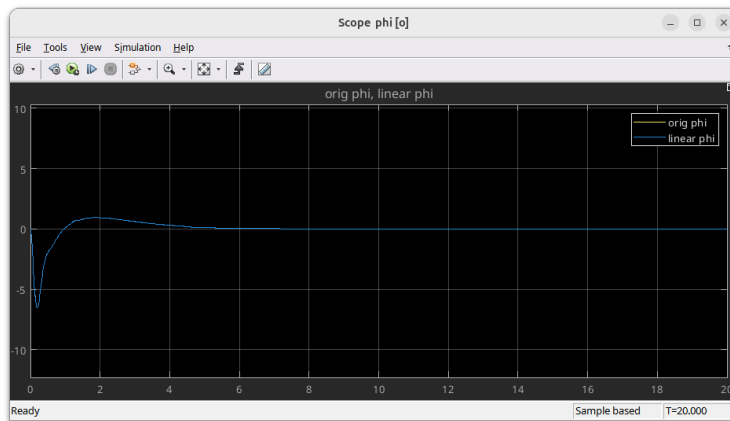
Z grafu 18 vidím, že kyvadlo během pohybu velmi kmitá. Toto kmitání bych chtěl utlumit, zvolím proto komplexně sdružené póly tak, aby měly stejnou imaginární hodnotu, jako matice A , ale větší imaginární hodnotu. Zbylé dva póly zmenším tak, abych vstupním napětím nedosahoval na saturaci, tedy $p = [-7 - 13.52i, -7 + 13.52i, -1.2, -1]$. Výsledek vidím na obrázcích 19 a 20.



Obrázek 18: Úhel kyvadla s neodladěnými póly



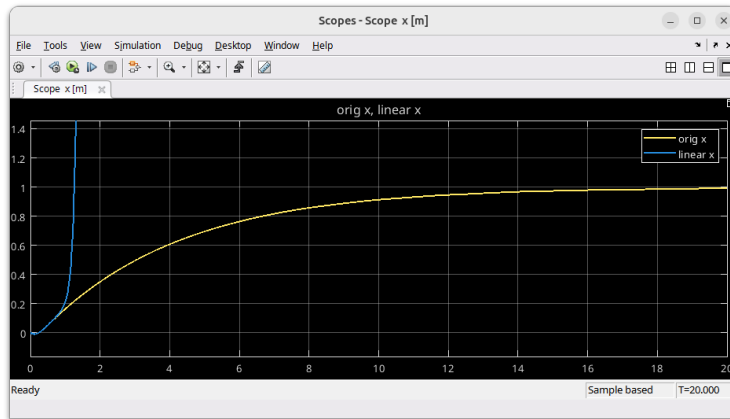
Obrázek 19: Poloha vozíku se stavovou zpětnou vazbou



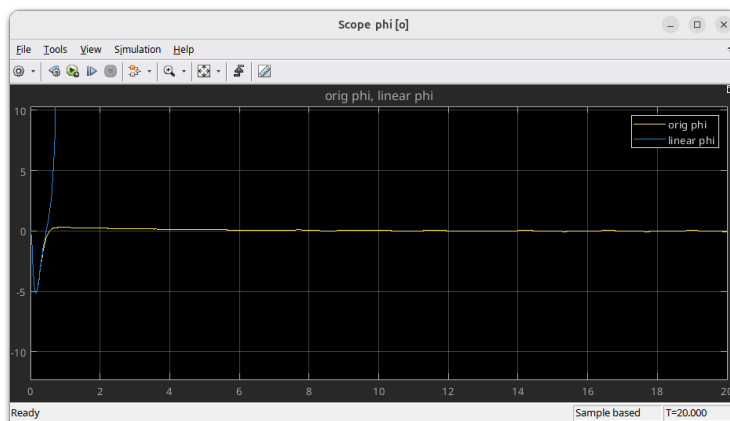
Obrázek 20: Úhel kyvadla se stavovou zpětnou vazbou

6.1.2. Horní poloha

S póly $p = [-19, -11 + 3i, -11 - 3i, -0.25]$ jsem schopen kyvadlo udržet v horní poloze. Rychlý pól v poloze -19 jsem zvolil tak, aby odpovídal pólu kyvadla v -18.98 , komplexně sdružené póly vyrovnávají padání kyvadla - jeho nestabilní pól v bodě 10.56 . Poslední pól jsem volil tak, aby se kyvadlo drželo v horní poloze s minimalní změnou úhlu a zároveň co nejdříve došlo na stanovenou pozici. Výsledky jsou na obrázcích 21 a 22.



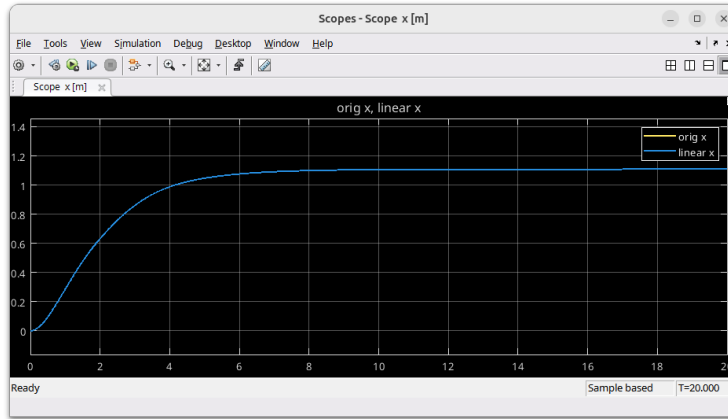
Obrázek 21:



Obrázek 22:

6.1.3. Problémy implementace s předfiltrem

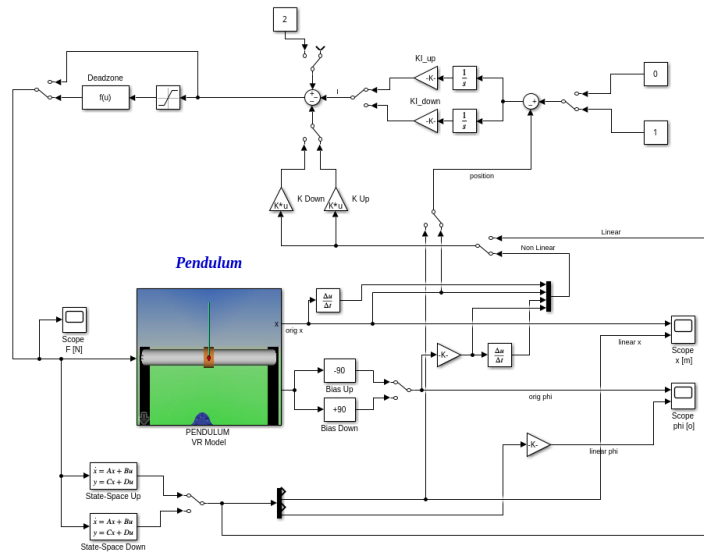
Implementace s předfiltrem je ovšem náchylná na poruchy, kvůli kterým vzniká ustálená odchylka, viz 23, kde jsem přidal step o hodnotě 1 v čase 2. Proto je potřeba implementovat integrální řízení.



Obrázek 23: Ustálená odchylka jako reakce na poruchu

6.2. Integrovní řešení stavové zpětné vazby

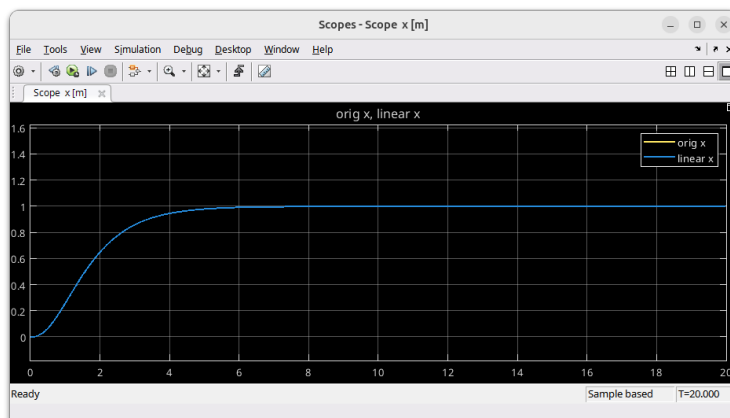
Pro integrovní řešení odstraním matici $\mathbf{M}$ a místo ní přidám integrátor za výstup polohy, výslednou zintegrovanou hodnotu vynásobím konstantou K_i a odečítám od výstupu z matice $\mathbf{K}$. Schéma je na obrázku 24.



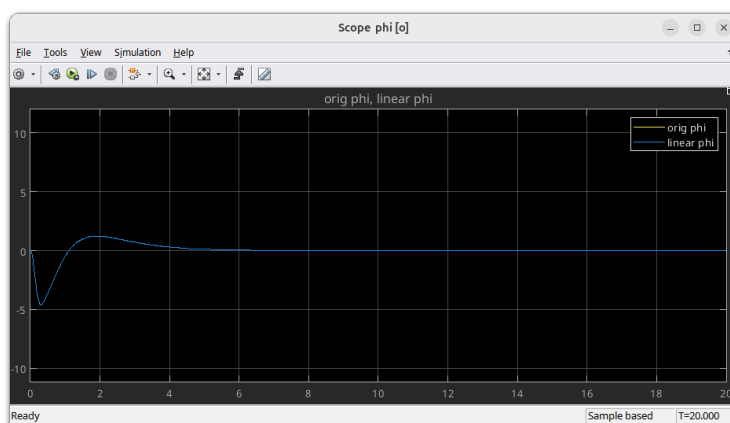
Obrázek 24: Zapojení stavové zpětné vazby s integrovním řešením v Simulinku

V Matlabu musím vypočítat hodnotu K_i . Nejdříve vytvořím složené matice $\mathbf{A}$ a $\mathbf{B}$: do matice $\mathbf{A}$ přidám řádek s $-\mathbf{C} = [0 \ -1 \ 0 \ 0]$, protože integrovní složkou chci doladit polohu. Zbytek složených matic doplním nulami. Matice $\mathbf{K}$, kterou opět dostanu funkcí `place`, bude mít tentokrát 5 hodnot - tou pátou bude K_i . Póly pro dolní polohu jsem zvolil jako $p = [-7 + 13.5160i, -7 - 13.5160i, -3, -2, -1]$ a pro horní polohu jako $p = [-19, -11 + 3i, -11 - 3i, -2, -0.5]$.

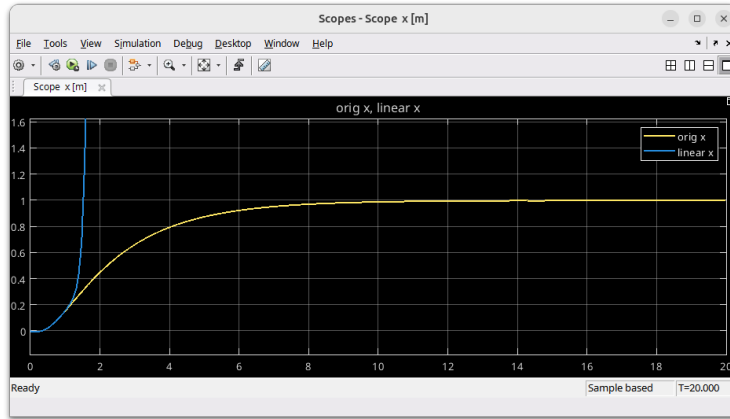
Integrální řešení je odolné i vůči konstantnímu působení síly některým směrem (mohu si to představit tak, že je rovina s kyvadlem nahnutá z kopce). Následují ukázky grafů s působící konstantní silou o hodnotě 2V pro dolní a horní polohu kyvadla.



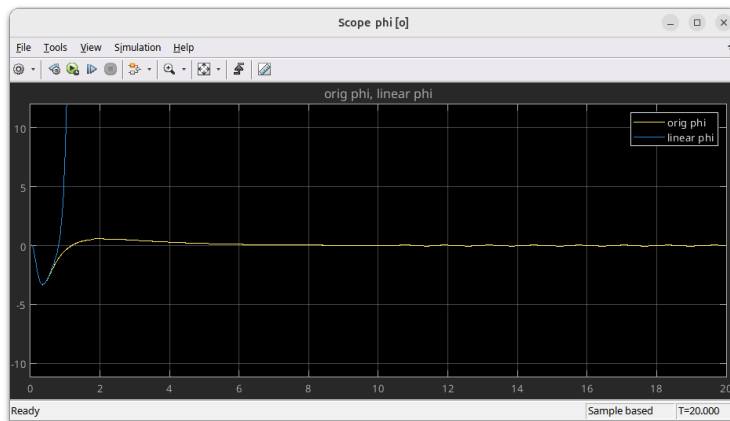
Obrázek 25: Stavová zpětná vazba pro dolní polohu: x



Obrázek 26: Stavová zpětná vazba pro dolní polohu: φ



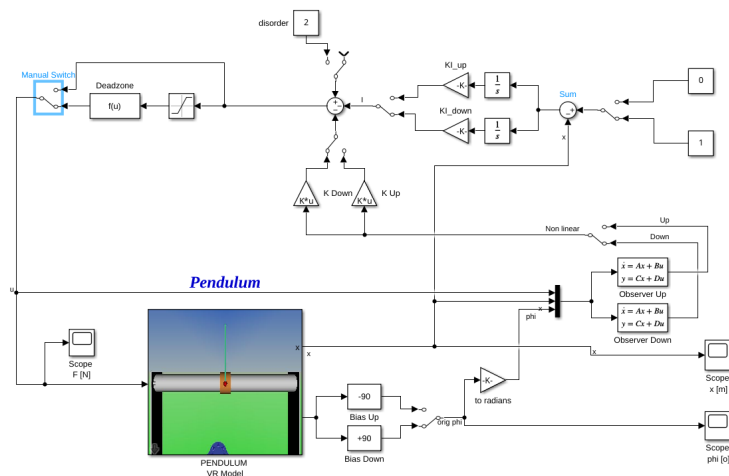
Obrázek 27: Stavová zpětná vazba pro horní polohu: x



Obrázek 28: Stavová zpětná vazba pro horní polohu: φ

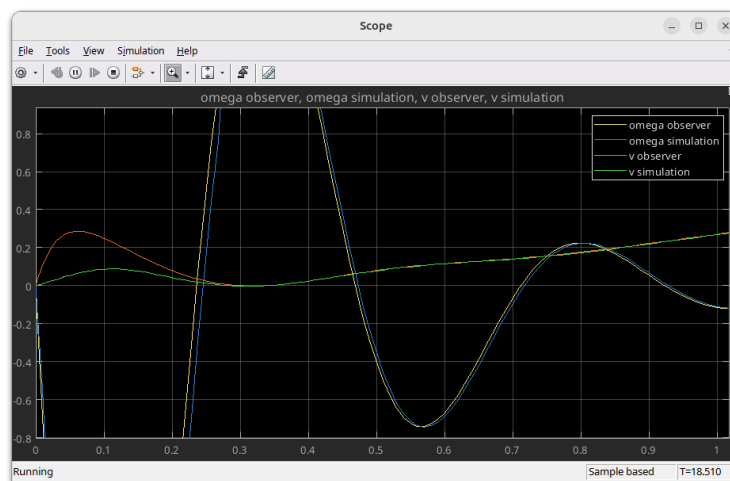
6.3. Pozorovatel

Obzvláště pro řízení reálného kyvadla je nutné implementovat pozorovatele, který bude určovat rychlost a úhlovou rychlost kyvadla - ty se totiž neměří senzorem. Pro pozorovatele na horní i dolní polohu vyberu póly (volil jsem je ± 2 krát rychlejší než rychlé póly regulátoru), a v Matlabu spočítám matici $\mathbf{L}$ pomocí $\mathbf{L} = \text{place}(\mathbf{A}', \mathbf{C}', \mathbf{p})'$. Následně zapojím do Simulinku jako na obrázku 29.



Obrázek 29: Zapojení pozorovatele v Simulinku

Zapojení pozorovatele jsem otestoval tak, že jsem odpojil zpětnou vazbu, do vozíku poslal sinusový signál, nastavil kyvadlo na -60° stupňů namísto -90° a sledoval, jestli pozorovatel dorovná signál z VR modelu, což se také stalo, viz obrázek 30.



Obrázek 30: Testování pozorovatele - graf rychlosti a úhlové rychlosti

7. Navazující úloha - práce se skutečným lineárním kyvadlem

Jako navazující úlohu jsem si vybral řízení lineárního kyvadla, které máme k dispozici v laboratoři. Práce probíhá podobně jako v simulaci, tedy vytvářím program skrz Simulink, který za běhu ovládá reálné kyvadlo a vrací mi polohu a úhel kyvadla. Oproti simulaci má kyvadlo ovšem jiné vlastnosti, kterým budu muset přizpůsobit své regulátory.

7.1. Identifikace systému

Nejdříve je nutné identifikovat systém. Přípustné hodnoty vstupního napětí jsou v rozsahu $u \in [-1, 1]$, ale reálné napětí je desetkrát větší, tedy $u \in [-10, 10][\text{V}]$. Deadzone jsem postupným zvyšováním napětí (viz obrázek TODO doplnit) identifikoval na hodnotu 0.14.

Reference

[1] Prof. Martin Hromčík. *Inverzni_kyvadlo_new.pdf*.